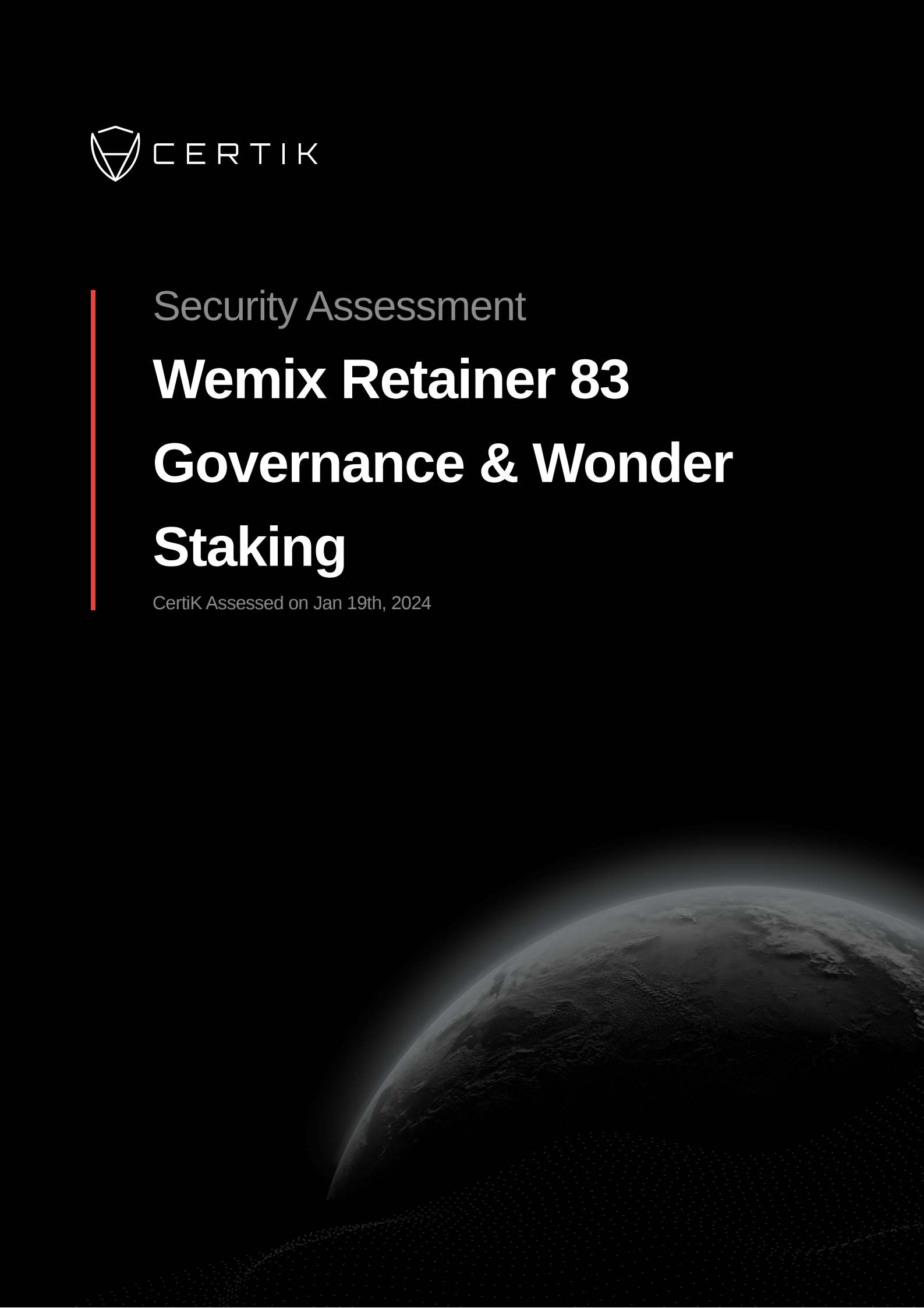
CERTIK

Security Assessment

Wemix Retainer 83 Governance & Wonder Staking

CertiK Assessed on Jan 19th, 2024



CertiK Assessed on Jan 19th, 2024

Wemix Retainer 83 Governance & Wonder Staking

The security assessment was prepared by CertiK, the leader in Web3.0 security.

Executive Summary

TYPES	ECOSYSTEM	METHODS
DeFi	EVM Compatible	Formal Verification, Manual Review, Static Analysis

LANGUAGE	TIMELINE	KEY COMPONENTS
Solidity	Delivered on 01/19/2024	N/A

CODEBASE

https://github.com/wemixarchive/Wonder-staking_private

https://github.com/wemixarchive/governance-contract

View All in Codebase Page

COMMITS

Wonder staking: 90399a33c720a6339f152f25f37028d4517b5e5b

Governance: 0fc5247b1751684a77c1fd87a6e76b6340de72c3

View All in Codebase Page

Vulnerability Summary



18	14	0	0	4	0
Total Findings	Resolved	Mitigated	Partially Resolved	Acknowledged	Declined

0	Critical		Critical risks are those that impact the safe functioning of a platform and must be addressed before launch. Users should not invest in any project with outstanding critical risks.
2	Major	2 Acknowledged	Major risks can include centralization issues and logical errors. Under specific circumstances, these major risks can lead to loss of funds and/or control of the project.
7	Medium	6 Resolved, 1 Acknowledged	Medium risks may not pose a direct risk to users' funds, but they can affect the overall functioning of a platform.
8	Minor	7 Resolved, 1 Acknowledged	Minor risks can be any of the above, but on a smaller scale. They generally do not compromise the overall integrity of the project, but they may be less efficient than other solutions.
1	Informational	1 Resolved	Informational errors are often recommendations to improve the style of the code or certain operations to fall within industry best practices. They usually do not affect the overall functioning of the code.

TABLE OF CONTENTS

WEMIX RETAINER 83 GOVERNANCE & WONDER STAKING

Summary

Executive Summary

Vulnerability Summary

Codebase

Audit Scope

Approach & Methods

Findings

CON-02 : Centralization Related Risks

WEM-01 : Centralized Control Of Contract Upgrade

BSI-01 : State Variables in Upgradeable Contracts are Initialized When Declared

CON-03 : Lack of Storage Gap in Upgradeable Contract

NCW-03 : No Upper Limit in function `setUnbondTime`

NCW-04 : Missing `nonReentrant` modifier in function `updateMP`

NCW-05 : Function `_deposit` allows user to claim reward on behalf of other accounts

NCW-09 : Function `removePool` will lead to lock of user deposit when the target pool is not empty

NCW-14 : Incorrect operation target address in `ncpWithdraw()`

NCP-01 : Missing Zero Address Validation

NCS-02 : Divide Before Multiply

NCW-07 : Missing checks for `feeRatioRequest` validation

NCW-10 : State variable `ncpToIdx[]` not reset in function `removePool`

NCW-12 : `updatePool` does not handle the `msg.value`

NCW-13 : Withdrawals recipient is the function caller instead of the specified address `to` in function `Withdraw()`

RWB-02 : Missing validation for rewarder existance

SIB-01 : Usage of `transfer()`

GIB-01 : `nonReentrant` Modifier can be Placed and Executed Before Other Modifiers In Functions

Optimizations

NCS-03 : Lack of Array Duplication Check

NCW-08 : Unused Mapping

RWB-01 : Unused State Variable

Formal Verification

Considered Functions And Scope

Verification Results

Appendix

Disclaimer

CODEBASE

WEMIX RETAINER 83 GOVERNANCE & WONDER STAKING

Repository

https://github.com/wemixarchive/Wonder-staking_private

https://github.com/wemixarchive/governance-contract

Commit

Wonder staking: 90399a33c720a6339f152f25f37028d4517b5e5b

Governance: 0fc5247b1751684a77c1fd87a6e76b6340de72c3

AUDIT SCOPE

WEMIX RETAINER 83 GOVERNANCE & WONDER STAKING

10 files audited ● 8 files with Acknowledged findings ● 2 files without findings

ID	Repo	File	SHA256 Checksum
SIB	wemixarchive/governance-contract	StakingImp.sol	87e5d8c14bf179f941cb6ad261b9f58c7b92f1de9638ceec8528c6c6f80ff657
NCP	wemixarchive/governance-contract	NCPExitImp.sol	b35f04f6c7f0fb0be5037ff194b6016fd846b9a4f04182df7a069645e70c1b56
GIB	wemixarchive/governance-contract	GovImp.sol	203259a8236d00a78e554405030cc1cd048f3894829899e563cb15fb7853d2ae
BSI	wemixarchive/governance-contract	storage/BallotStorageImp.sol	cdab2c8f8f31debdba90ebcfe1fc1d62aa9e2babae99fc3a35e30c868f9960f5
NCS	wemixarchive/Wonder-staking_private	gateway/NCPStakingGateway.sol	78f6775dec43b057c3b167d335f9fa905138fcd1008a33900008ff7cd19f7412
NCW	wemixarchive/Wonder-staking_private	NCPStaking.sol	ed02f8aa837ecfee84c346503acf881f092233d87ffd0853864b6b939e5313b6
RWB	wemixarchive/Wonder-staking_private	Rewarder.sol	044cdd2dea16a8ceb41ba5d2f3200301447b71aa35e0ae182bbebc0f54adc9c6
WNF	wemixarchive/Wonder-staking_private	withdrawalToken/WithdrawalNFT.sol	36980a22d6d450c8b04ea8b7e2070b1db6eca75fa7540b666f17a8458c2a44ee
NCE	wemixarchive/governance-contract	NCPExit.sol	d3306e62784891a35ccfe14d91b9aaffdf71a6d82136ccdbb32068bb2230ee4a
BSB	wemixarchive/governance-contract	storage/BallotStorage.sol	ded5225059a032264990daf0d6a9c2850e00b0d436e463c87af7dc00ef8764db

APPROACH & METHODS

WEMIX RETAINER 83 GOVERNANCE & WONDER STAKING

This report has been prepared for Wemix to discover issues and vulnerabilities in the source code of the Wemix Retainer 83 Governance & Wonder Staking project as well as any contract dependencies that were not part of an officially recognized library. A comprehensive examination has been performed, utilizing Static Analysis and Manual Review techniques.

The auditing process pays special attention to the following considerations:

- Testing the smart contracts against both common and uncommon attack vectors.
- Assessing the codebase to ensure compliance with current best practices and industry standards.
- Ensuring contract logic meets the specifications and intentions of the client.
- Cross referencing contract structure and implementation against similar smart contracts produced by industry leaders.
- Thorough line-by-line manual review of the entire codebase by industry experts.

The security assessment resulted in findings that ranged from critical to informational. We recommend addressing these findings to ensure a high level of security standards and industry practices. We suggest recommendations that could better serve the project from the security perspective:

- Testing the smart contracts against both common and uncommon attack vectors;
- Enhance general coding practices for better structures of source codes;
- Add enough unit tests to cover the possible use cases;
- Provide more comments per each function for readability, especially contracts that are verified in public;
- Provide more transparency on privileged activities once the protocol is live.

FINDINGS

WEMIX RETAINER 83 GOVERNANCE & WONDER STAKING



18	0	2	7	8	1
Total Findings	Critical	Major	Medium	Minor	Informational

This report has been prepared to discover issues and vulnerabilities for Wemix Retainer 83 Governance & Wonder Staking. Through this audit, we have uncovered 18 issues ranging from different severity levels. Utilizing the techniques of Static Analysis & Manual Review to complement rigorous manual code reviews, we discovered the following findings:

ID	Title	Category	Severity	Status
CON-02	**Centralization Related Risks**	**Centralization**	**Major**	**Acknowledged**
WEM-01	**Centralized Control Of Contract Upgrade**	**Centralization**	**Major**	**Acknowledged**
BSI-01	State Variables In Upgradeable Contracts Are Initialized When Declared	Logical Issue	Medium	Resolved
CON-03	Lack Of Storage Gap In Upgradeable Contract	Design Issue	Medium	Acknowledged
NCW-03	No Upper Limit In Function `setUnbondTime`	Logical Issue	Medium	Resolved
NCW-04	Missing `nonReentrant` Modifier In Function `updateMP`	Logical Issue	Medium	Resolved
NCW-05	Function `_deposit` Allows User To Claim Reward On Behalf Of Other Accounts	Logical Issue	Medium	Resolved
NCW-09	Function `removePool` Will Lead To Lock Of User Deposit When The Target Pool Is Not Empty	Logical Issue	Medium	Resolved
NCW-14	Incorrect Operation Target Address In `ncpWithdraw()`	Logical Issue	Medium	Resolved
NCP-01	Missing Zero Address Validation	Volatile Code	Minor	Resolved

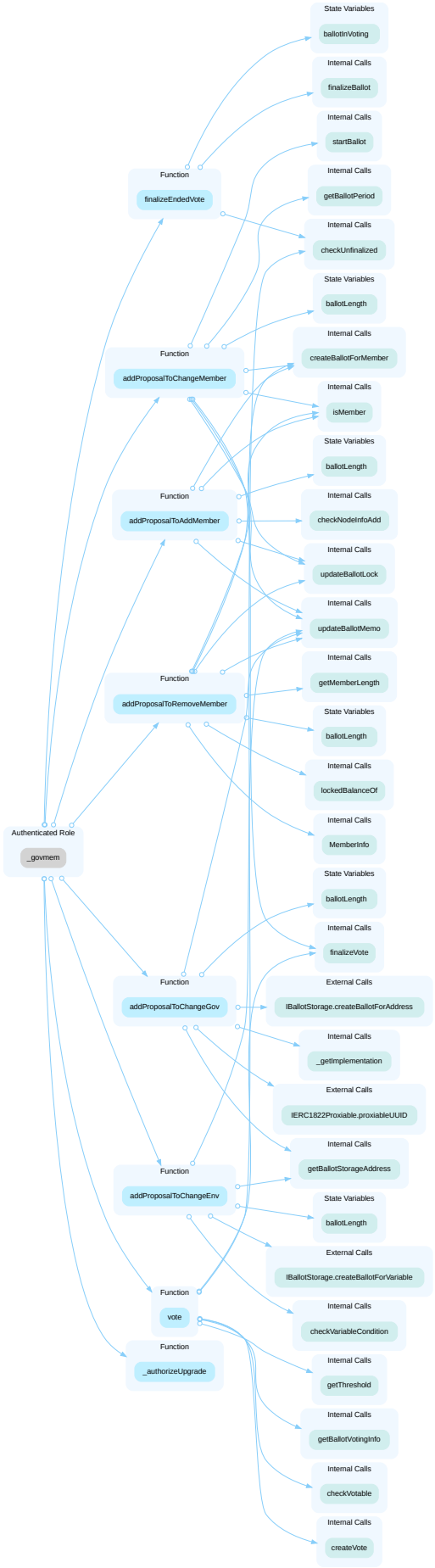
ID	Title	Category	Severity	Status
NCS-02	Divide Before Multiply	Incorrect Calculation	Minor	Acknowledged
NCW-07	Missing Checks For `feeRatioRequest` Validation	Logical Issue	Minor	Resolved
NCW-10	State Variable `ncpToIdx[]` Not Reset In Function `removePool`	Logical Issue	Minor	Resolved
NCW-12	`updatePool` Does Not Handle The `msg.value`	Logical Issue	Minor	Resolved
NCW-13	Withdrawals Recipient Is The Function Caller Instead Of The Specified Address `to` In Function `Withdraw()`	Logical Issue	Minor	Resolved
RWB-02	Missing Validation For Rewarder Existance	Logical Issue	Minor	Resolved
SIB-01	Usage Of `transfer()`	Volatile Code	Minor	Resolved
GIB-01	`nonReentrant` Modifier Can Be Placed And Executed Before Other Modifiers In Functions	Volatile Code	Informational	Resolved

CON-02 | CENTRALIZATION RELATED RISKS

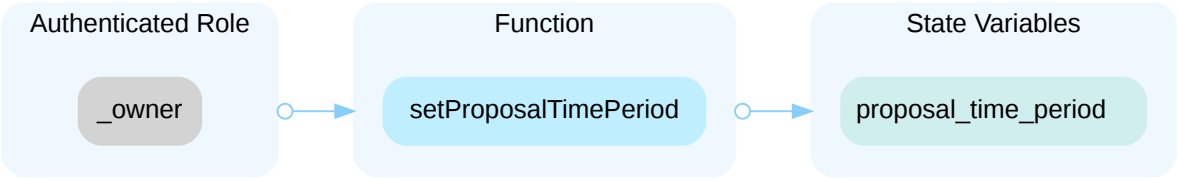
Category	Severity	Location	Status
Centralization	**Major**	**GovImp.sol (v1): 272, 299, 355, 419, 449, 484, 550, 1188, 1211, 1276; NCPExitImp.sol (v1): 65, 72, 97, 105, 110, 117; StakingImp.sol (v1): 84, 110, 142, 146, 179, 211, 269, 281, 288, 328, 337, 362**	**Acknowledged**

Description

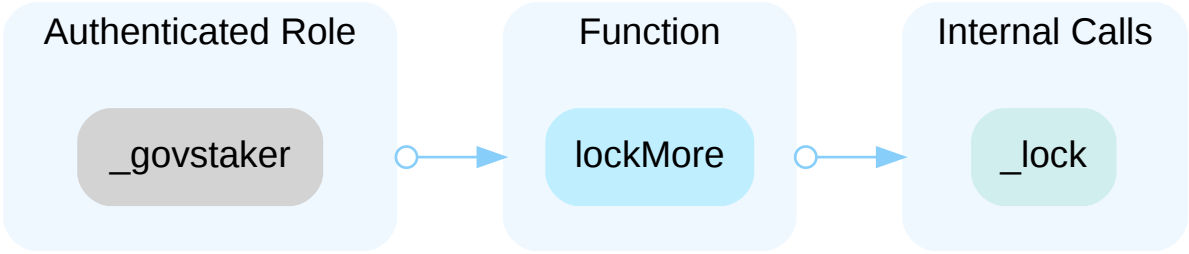
In the contract `GovImp` the role `_govmem` has authority over the functions shown in the diagram below. Any compromise to the `_govmem` account may allow the hacker to take advantage of this authority and add proposals, vote, and finalize the end vote.


Authenticated Role
_govmem
Function
finalizeEndedVote
Function
addProposalToChangeMember
Function
addProposalToAddMember
Function
addProposalToRemoveMember
Function
addProposalToChangeGov
Function
addProposalToChangeEnv
Function
vote
Function
_authorizeUpgrade
State Variables
ballotInVoting
Internal Calls
finalizeBallot
Internal Calls
startBallot
Internal Calls
getBallotPeriod
Internal Calls
checkUnfinalized
State Variables
ballotLength
Internal Calls
createBallotForMember
Internal Calls
isMember
State Variables
ballotLength
Internal Calls
checkNodeInfoAdd
Internal Calls
updateBallotLock
Internal Calls
updateBallotMemo
Internal Calls
getMemberLength
State Variables
ballotLength
Internal Calls
lockedBalanceOf
Internal Calls
MemberInfo
State Variables
ballotLength
Internal Calls
finalizeVote
External Calls
IBallotStorage.createBallotForAddress
Internal Calls
_getImplementation
External Calls
IERC1822Proxiable.proxiableUUID
Internal Calls
getBallotStorageAddress
State Variables
ballotLength
External Calls
IBallotStorage.createBallotForVariable
Internal Calls
checkVariableCondition
Internal Calls
getThreshold
Internal Calls
getBallotVotingInfo
Internal Calls
checkVotable
Internal Calls
createVote

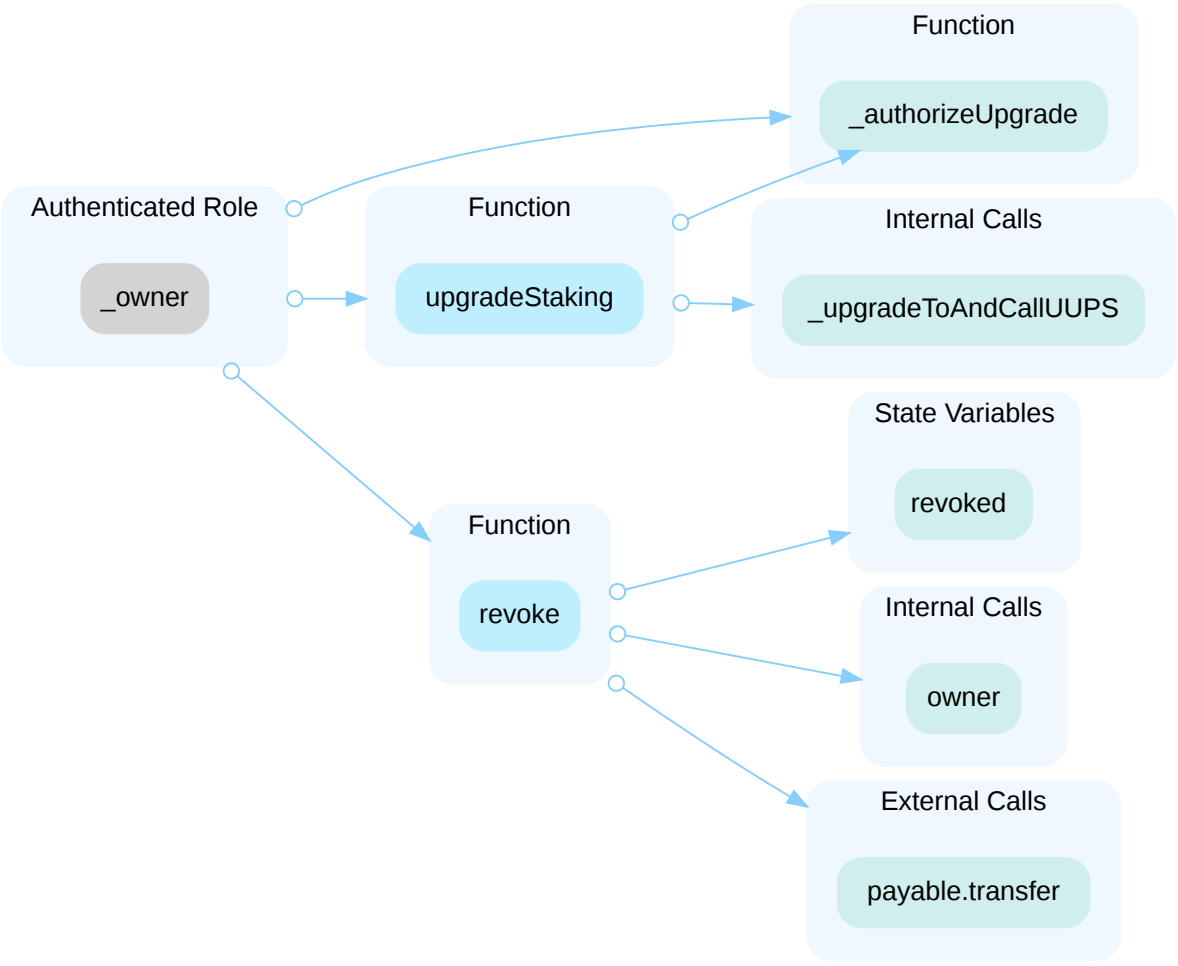
In the contract `GovImp` the role `_owner` has authority over the functions shown in the diagram below. Any compromise to the `_owner` account may allow the hacker to take advantage of this authority and can re-init and set `posalTimePeriod`.



In the contract `StakingImp` the role `_govstaker` has authority over the functions shown in the diagram below. Any compromise to the `_govstaker` account may allow the hacker to take advantage of this authority and lock more amounts.



In the contract `StakingImp` the role `_owner` has authority over the functions shown in the diagram below. Any compromise to the `_owner` account may allow the hacker to take advantage of this authority and revoke the staking contract, upgrade the staking contract, and set the NCP staking address.



In the contract `NCPExitImp` the role `_administrator` has authority over the functions

- withdrawForAdministrator()
- getAvailableAmountForAdministrator()

Any compromise to the `_administrator` account may allow the hacker to take advantage of this authority and withdraw tokens to any address and get the available amount.

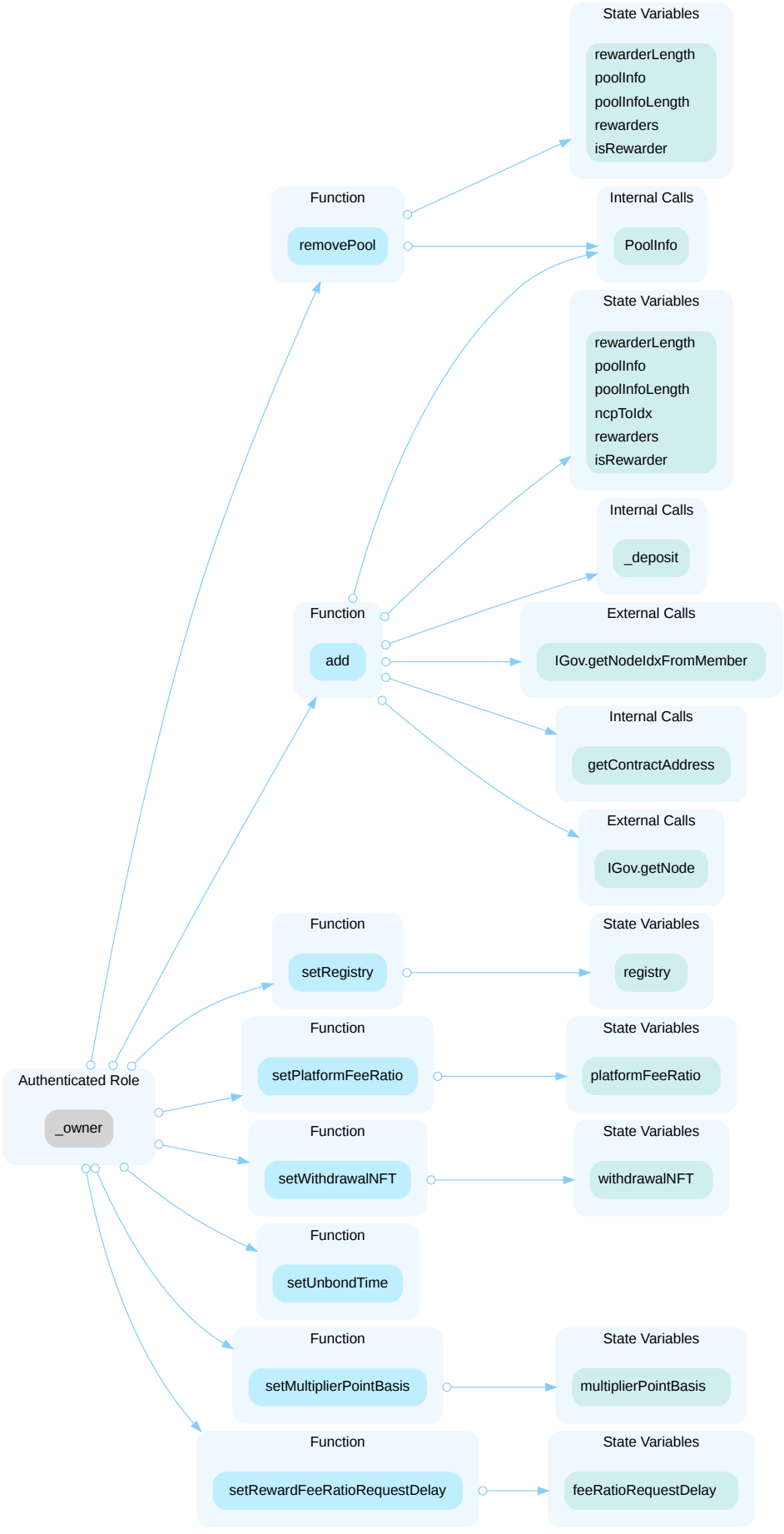
In the contract `NCPExitImp` the role `_administratorSetter` has authority over the functions

- setAdministrator()
- setAdministratorSetter()

Any compromise to the `_administratorSetter` account may allow the hacker to take advantage of this authority and set the administrator and administrator setter address.

Any compromise to the `_owner` of the contract `NCPExitImp` may allow the hacker to take advantage of this authority and upgrade the `NCPExitImp` contract.

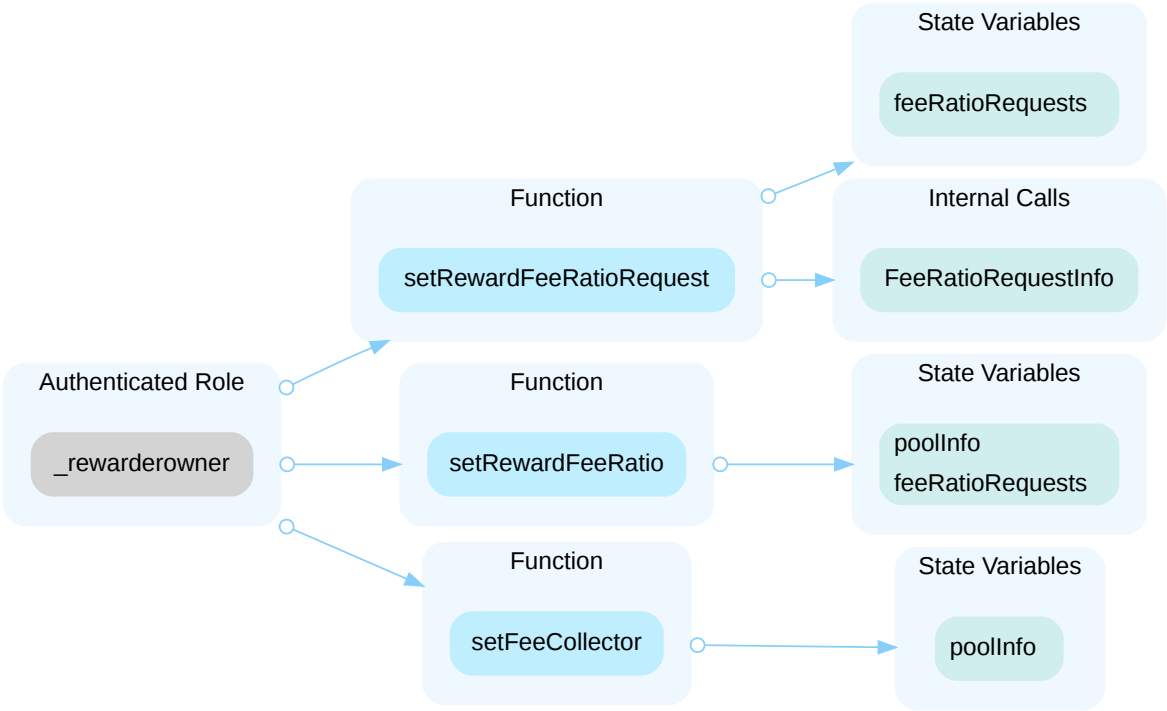
In the contract `NCPStaking` the role `_owner` has authority over the functions shown in the diagram below.


State Variables
rewarderLength
poolInfo
poolInfoLength
rewarders
isRewarder
Function
removePool
Internal Calls
PoolInfo
State Variables
rewarderLength
poolInfo
poolInfoLength
ncpToIdx
rewarders
isRewarder
Internal Calls
_deposit
Function
add
External Calls
IGov.getNodeIdxFromMember
Internal Calls
getContractAddress
External Calls
IGov.getNode
Function
setRegistry
State Variables
registry
Function
setPlatformFeeRatio
State Variables
platformFeeRatio
Authenticated Role
_owner
Function
setWithdrawalNFT
State Variables
withdrawalNFT
Function
setUnbondTime
Function
setMultiplierPointBasis
State Variables
multiplierPointBasis
Function
setRewardFeeRatioRequestDelay
State Variables
feeRatioRequestDelay

Any compromise to the `_owner` account may allow the hacker to take advantage of this authority and can call multiple functions such as `removePool`, `setPlatformFeeRatio`, `setWithdrawalNFT`, `addPool`, and so on. The hacker can set the platform fee ratio to 100% or remove an existing pool to reset all user rewards. If the hacker removes an active pool, users will not be able to withdraw their stakings. Besides, the `add` function allows the contract owner to perform the initial deposit without any cost.**

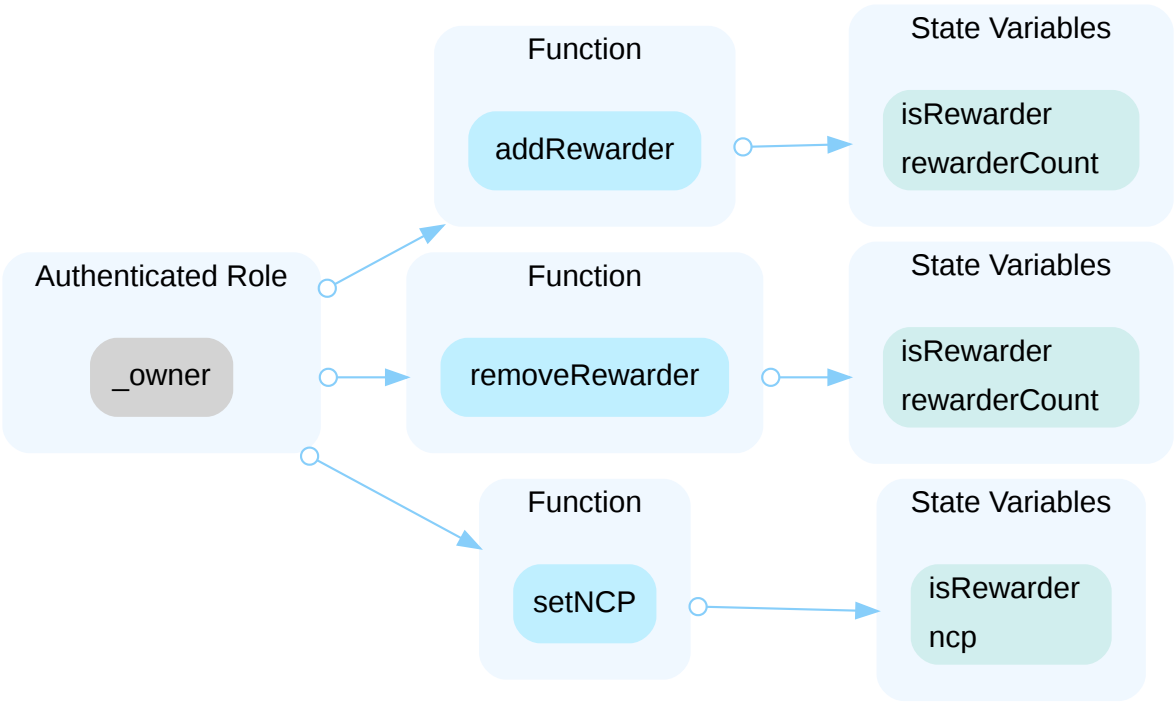
Once the owner role gets compromised, the attacker can withdraw funds from `GovernanceStaking` by adding a new pool and then withdraw.

In the contract `NCPStaking` the role `_rewarderowner` has authority over the functions shown in the diagram below.



Any compromise to the `_rewarderowner` account may allow the hacker to take advantage of this authority and can change `rewardFeeRatio`.

In the contract `Rewarder` the role `_owner` has authority over the functions shown in the diagram below.



Any compromise to the `_owner` account may allow the hacker to take advantage of this authority and add/remove rewarder addresses and NCP.

Recommendation

The risk describes the current project design and potentially makes iterations to improve in the security operation and level of decentralization, which in most cases cannot be resolved entirely at the present stage. We advise the client to carefully manage the privileged account's private key to avoid any potential risks of being hacked. In general, we strongly recommend centralized privileges or roles in the protocol be improved via a decentralized mechanism or smart-contract-based accounts with enhanced security practices, e.g., multisignature wallets.

Indicatively, here are some feasible suggestions that would also mitigate the potential risk at a different level in terms of short-term, long-term and permanent:

Short Term:

Timelock and Multi sign (⅔, ⅗) combination *mitigate* by delaying the sensitive operation and avoiding a single point of key management failure.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;\

AND

- Assignment of privileged roles to multi-signature wallets to prevent a single point of failure due to the private key compromised;\

AND

- A medium/blog link for sharing the timelock contract and multi-signers addresses information with the public audience.

Long Term:

Timelock and DAO, the combination, *mitigate* by applying decentralization and transparency.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;\

AND

- Introduction of a DAO/governance/voting module to increase transparency and user involvement.\

AND

- A medium/blog link for sharing the timelock contract, multi-signers addresses, and DAO information with the public audience.

Permanent:

Renouncing the ownership or removing the function can be considered *fully resolved*.

- Renounce the ownership and never claim back the privileged roles.\

OR

- Remove the risky functionality.

Alleviation

[Wemix, 01/18/2024]: We will use a time-lock and a multi-sig contract with Governance members.

[CertiK, 01/18/2024]: It should be noted that the centralization risk issue still exists. CertiK strongly encourages the project team to periodically revisit the private key security management of all addresses related to centralized roles.

WEM-01 | CENTRALIZED CONTROL OF CONTRACT UPGRADE

Category	Severity	Location	Status
Centralization	**Major**	**NCPStaking.sol (v1): 10; Rewarder.sol (v1): 10; gateway/NCPStakingGateway.sol (v1): 10; withdrawalToken/WithdrawalNFT.sol (v1): 10; GovChecker.sol (v1): 12; GovImp.sol (v1): 17, 20; NCPExitImp.sol (v1): 15, 16; StakingImp.sol (v1): 16, 16; storage/BallotStorageImp.sol (v1): 10; storage/EnvStorageImp.sol (v1): 10**	**Acknowledged**

Description

In the above contracts, the role `owner` has the authority to update the implementation contract behind the proxy contract.

Any compromise to the `owner` account may allow a hacker to take advantage of this authority and change the implementation contract which is pointed by proxy and therefore execute potential malicious functionality in the implementation contract.

Recommendation

We recommend that the team make efforts to restrict access to the admin of the proxy contract. A strategy of combining a time-lock and a multi-signature (⅔, ⅗) wallet can be used to prevent a single point of failure due to a private key compromise. In addition, the team should be transparent and notify the community in advance whenever they plan to migrate to a new implementation contract.

Here are some feasible short-term and long-term suggestions that would mitigate the potential risk to a different level and suggestions that would permanently fully resolve the risk.

Short Term:

A combination of a time-lock and a multi signature (⅔, ⅗) wallet mitigate the risk by delaying the sensitive operation and avoiding a single point of key management failure.

- A time-lock with reasonable latency, such as 48 hours, for awareness of privileged operations;
  AND
- Assignment of privileged roles to multi-signature wallets to prevent a single point of failure due to a private key compromised;
  AND
- A medium/blog link for sharing the time-lock contract and multi-signers addresses information with the community.

For remediation and mitigated status, please provide the following information:

- Provide the deployed time-lock address.
- Provide the **gnosis** address with **ALL** the multi-signer addresses for the verification process.
- Provide a link to the **medium/blog** with all of the above information included.

Long Term:

A combination of a time-lock on the contract upgrade operation and a DAO for controlling the upgrade operation mitigate the contract upgrade risk by applying transparency and decentralization.

- A time-lock with reasonable latency, such as 48 hours, for community awareness of privileged operations;
  AND
- Introduction of a DAO, governance, or voting module to increase decentralization, transparency, and user involvement;
  AND
- A medium/blog link for sharing the time-lock contract, multi-signers addresses, and DAO information with the community.

For remediation and mitigated status, please provide the following information:

- Provide the deployed time-lock address.
- Provide the **gnosis** address with **ALL** the multi-signer addresses for the verification process.
- Provide a link to the **medium/blog** with all of the above information included.

Permanent:

Renouncing ownership of the `admin` account or removing the upgrade functionality can *fully* resolve the risk.

- Renounce the ownership and never claim back the privileged role;
  OR
- Remove the risky functionality.

Note: we recommend the project team consider the long-term solution or the permanent solution. The project team shall make a decision based on the current state of their project, timeline, and project resources.

Alleviation

[Wemix, 01/18/2024]: We will use a time-lock and a multi-sig contract with Governance members.

[CertiK, 01/18/2024]: It should be noted that the centralization risk issue still exists. CertiK strongly encourages the project

team to periodically revisit the private key security management of all addresses related to centralized roles.

BSI-01 STATE VARIABLES IN UPGRADEABLE CONTRACTS ARE INITIALIZED WHEN DECLARED

Category	Severity	Location	Status
Logical Issue	Medium	storage/BallotStorageImp.sol (v1): 131	Resolved

Description

State variables initialized when declared are equivalent to initializing them inside the constructor. Therefore, initializing state variables when declared in an upgradeable contract has no actual effect since the constructor of an upgradeable contract is never called.

Recommendation

We recommend initializing state variables in an initializer function if necessary to avoid unexpected behavior and confusion.

Alleviation

The client revised the code and resolved this issue in commit : 0fc5247b1751684a77c1fd87a6e76b6340de72c3

CON-03 | LACK OF STORAGE GAP IN UPGRADEABLE CONTRACT

Category	Severity	Location	Status
Design Issue	Medium	NCPExitImp.sol (v1): 13; StakingImp.sol (v1): 16; abstract/AEnvStorage.sol (v1): 8; abstract/AGov.sol (v1): 8; storage/BallotStorageImp.sol (v1): 10	Acknowledged

Description

There is no storage gap preserved in the logic contract `GovChecker`. Any logic contract that acts as a base contract inherited by other upgradeable contracts should have a reasonable size of storage gap preserved for potential new state variables introduced in future upgrades. Otherwise, there could be potential storage slot collision.

Recommendation

It is recommended to include a storage gap of a reasonable size preserved in the logic contract in case that new state variables are introduced in future upgrades. For more information, please refer to: https://docs.openzeppelin.com/contracts/4.x/upgradeable#storage_gaps.

Alleviation

[Wemix, 01/18/2024]: Issue acknowledged. I won't make any changes for the current version.

NCW-03 | NO UPPER LIMIT IN FUNCTION `setUnbondTime`

Category	Severity	Location	Status
Logical Issue	Medium	NCPStaking.sol (v1): 419~424	Resolved

Description

There are no upper boundaries for `setUnbondTime` which is used to determin when users can withdraw. It is possible to set the unbond time up to any arbitrary amount.

In this case users will never be able to withdraw their stakings.

Recommendation

We recommend adding reasonable boundaries for the unbond time.

Alleviation

Issue resolved. Changes have been reflected in the commit hash: https://github.com/wemixarchive/Wonder-staking_private/commit/90399a33c720a6339f152f25f37028d4517b5e5b

NCW-04 MISSING `nonReentrant` MODIFIER IN FUNCTION `updateMP`

Category	Severity	Location	Status
Logical Issue	Medium	NCPStaking.sol (v1): 750~775	Resolved

Description

The `nonReentrant` modifier is missing in the public function `updateMP`. The absence of this modifier exposes the smart contract to potential reentrancy vulnerabilities, which could result in unauthorized state changes or reward amount manipulation.

Recommendation

We recommend implementing the reentrancy lock to the function `updateMP`.

Alleviation

Issue resolved. Changes have been reflected in the commit hash: https://github.com/wemixarchive/Wonder-staking_private/commit/90399a33c720a6339f152f25f37028d4517b5e5b

NCW-05 FUNCTION `_deposit` ALLOWS USER TO CLAIM REWARD ON BEHALF OF OTHER ACCOUNTS

Category	Severity	Location	Status
Logical Issue	Medium	NCPStaking.sol (v1): 828~855	Resolved

Description

In the current implementation of the `_deposit(pid, amount, to, claimReward)` function, `_harvest` is invoked for the destination account ("to" parameter), leading to the disbursement of pending rewards to that account.

```
    function deposit(
        uint256 pid,
        uint256 amount,
        address payable to,
        bool claimReward,
        bool comp
    ) external payable nonReentrant whenNotLock(pid) {
        require(pid != 37 && pid != 40, "STAKING: 37, 40 is stoped");
        require(
            msg.value == amount,
            "STAKING: Wrong amount"
        );
        require(
            msg.sender != poolInfo[ncpToIdx[msg.sender]].ncp,
            "STAKING: NCP cannot deposit"
        );

        _deposit(pid, amount, to, claimReward);
        ...

```

In function `_deposit()`:

```
    if (user.amount > 0) {
            _harvest(pid, to, to, claimReward, true);
            // console.log("after harvest",pool.activatedMP);
    }
```

This raises a potential security issue in a situation where, for example, User A executes `deposit()` with the destination address 'to' set as User B and the `claimReward` parameter set to `True`. In this case, the reward associated with User B would be claimed and transferred to User B, but this process takes place without requiring explicit authorization from User B. This could potentially allow User A to interfere with User B's rewards claim under certain circumstances.

Recommendation

We would advise the development team to clarify whether the aforementioned cases are intended design.

Alleviation

Issue resolved. Changes have been reflected in the commit hash: https://github.com/wemixarchive/Wonder-staking_private/commit/90399a33c720a6339f152f25f37028d4517b5e5b

NCW-09 FUNCTION `removePool` WILL LEAD TO LOCK OF USER DEPOSIT WHEN THE TARGET POOL IS NOT EMPTY

Category	Severity	Location	Status
Logical Issue	Medium	NCPStaking.sol (v1): 326~328	Resolved

Description

Stakers cannot withdraw their deposits when the pool is removed because the check in `updatePool` will revert the transaction.

```
1  require(pool.lastRewardBlock != 0, "STAKING: Pool does not exist");
```

Any withdraw attemps after pool removal will revert. In this case, users will lose unclaimed stakings and rewards.

Recommendation

We recommend the team implement proper checks to ensure all stakers have their deposit withdrawn before removing an existing pool.

Alleviation

Issue resolved. Changes have been reflected in the commit hash: https://github.com/wemixarchive/Wonder-staking_private/commit/90399a33c720a6339f152f25f37028d4517b5e5b

NCW-14 INCORRECT OPERATION TARGET ADDRESS IN `ncpWithdraw()`

Category	Severity	Location	Status
Logical Issue	Medium	NCPStaking.sol (v1): 915~920	Resolved

Description

The caller (msg.sender) of function `ncpWithdraw` is the `GovStaking` contract, while the actual action should be attributed to the NCP address that initialized the function call in the contract `GovStaking`, rather than the `GovStaking` itself (msg.sender).

In the following code snippet, the usages of `msg.sender` should all be replaced with the NCP address that initiated this function call.

```
904         (pool, mpInfo) = _updateMP(pid, msg.sender);
```

```
915
       if (IGov(getContractAddress("GovernanceContract")).isStaker(msg.sender)) {
916             require(
917
                user.amount >=
IGov(getContractAddress("GovernanceContract")).getMinStaking(),
918                 "STAKING: Insufficient amount to withdraw"
919             );
920         }
921
```

```
936         userMPInfo[pid][msg.sender] = mpInfo;
```

```
942
943         emit WithdrawRequest(
944             msg.sender,
945             pid,
946             pool.name,
947             pool.ncp,
948             amount,
949             to,
950             userRewardAmount
951         );
```

Recommendation

We recommend the team replace the usage of `msg.sender` with the NCP address that initiated this function call in function `ncpWithdraw()`.

Alleviation

Issue resolved. Changes have been reflected in the commit hash: https://github.com/wemixarchive/Wonder-staking_private/commit/90399a33c720a6339f152f25f37028d4517b5e5b

NCP-01 | MISSING ZERO ADDRESS VALIDATION

Category	Severity	Location	Status
Volatile Code	Minor	NCPExitImp.sol (v1): 66, 73	Resolved

Description

Addresses are not validated before assignment or external calls, potentially allowing the use of zero addresses and leading to unexpected behavior or vulnerabilities. For example, transferring tokens to a zero address can result in a permanent loss of those tokens.

Recommendation

It is recommended to add a zero-check for the passed-in address value to prevent unexpected errors.

Alleviation

The client revised the code and resolved this issue in commit : a2ce95f325693f9d07dd6b35c89d4f68d6ce7172

NCS-02 | DIVIDE BEFORE MULTIPLY

Category	Severity	Location	Status
Incorrect Calculation	Minor	gateway/NCPStakingGateway.sol (v1): 59, 74~77, 289, 291, 292, 309~310, 311~314, 332~333, 337~340	Acknowledged

Description

Performing integer division before multiplication truncates the low bits, losing the precision of calculation.

```
59          uint256 poolTotalRewardAPR = ((totalRewardAPR) * 1 ether) /
 govTotalLockedBalance;
```

```
 74              uint256 apr =
 75                  ((poolTotalRewardAPR * (10000 - platformFeeRatio))  *
 76                      (10000 - pool.feeRatio)) / 10000 /
 77                  10000;
```

```
289          uint256 x = computePoolAPR(pid) / SECONDS_PER_YEAR * 10**18;
```

```
291          uint256 x2 = x * x / BASE_ETHER;
```

```
292          uint256 x3 = x2 * x / BASE_ETHER;
```

```
309              uint256 poolTotalRewardAPR = (getTotalRewardAPR() * ratio * 1 ether
) / 10000 /
310                  stakedLP;
```

```
311              apr =
312                  (((poolTotalRewardAPR * (10000 - platformFeeRatio))) *
313                      (10000 - pool.feeRatio))  / 10000 /
314                  10000;
```

```
332              uint256 poolTotalRewardAPR = (getTotalRewardAPR() * ratio * 1 ether
) / 10000 /
333                  stakedLP;
```

```
337             apr =
338                 ((poolTotalRewardAPR * (10000 - platformFeeRatio)) *
339                     (10000 - feeRatio)) / 10000 /
340                 10000;
```

Recommendation

We recommend applying multiplication before division to avoid loss of precision.

Alleviation

[Wemix, 01/18/2014]: When calculating data, it has a precision of 10**18, so there is no significant impact on the calculation precision of the token quantity.

NCW-07 | MISSING CHECKS FOR `feeRatioRequest` VALIDATION

Category	Severity	Location	Status
Logical Issue	Minor	NCPStaking.sol (v1): 401~417	Resolved

Description

In the current implementation, function `setRewardFeeRatio` will reset `feeRatioRequests` after changes been made.

However, if the `rewarderOwner` does not propose a valid `setRewardFeeRatioRequest` and calls the `setRewardFeeRatio` directly, the request (`feeRatioRequests[pid].ratio = 0 && feeRatioRequests[pid].requestBlockNumber`) will also be considered as a valid request thus reset the fee ratio to zero.

Recommendation

We recommend the team to check the `requestBlockNumber` is not zero before making the fee ratio change in the function `setRewardFeeRatio()` .

Alleviation

Issue resolved. Changes have been reflected in the commit hash: https://github.com/wemixarchive/Wonder-staking_private/commit/90399a33c720a6339f152f25f37028d4517b5e5b

NCW-10 STATE VARIABLE `ncpToIdx[]` NOT RESET IN FUNCTION `removePool`

Category	Severity	Location	Status
Logical Issue	Minor	NCPStaking.sol (v1): 353~355	Resolved

Description

When removing a staking pool, all of the pool-related state variables should be reset.

However, in the current implementation, the state variable `ncpToIdx[]` is not reset properly, which in some cases will prevent creating another pool for the same ncp participant.

Recommendation

We recommend the team reset the state variable `ncpToIdx[]` when removing an existing ncp pool.

Alleviation

Issue resolved. Changes have been reflected in the commit hash: https://github.com/wemixarchive/Wonder-staking_private/commit/90399a33c720a6339f152f25f37028d4517b5e5b

NCW-12 | `updatePool` DOES NOT HANDLE THE `msg.value`

Category	Severity	Location	Status
Logical Issue	Minor	NCPStaking.sol (v1): 708~744	Resolved

Description

In the current implementation, function `updatePool` has the `payable` attribute but failed to handle the input native token.

These part of native token will be locked in the contract.

Recommendation

We recommend the team removing the `payable` attribute from the `updatePool` function.

Alleviation

Issue resolved. Changes have been reflected in the commit hash: https://github.com/wemixarchive/Wonder-staking_private/commit/90399a33c720a6339f152f25f37028d4517b5e5b

NCW-13 WITHDRAWALS RECIPIENT IS THE FUNCTION CALLER INSTEAD OF THE SPECIFIED ADDRESS `to` IN FUNCTION `Withdraw()`

Category	Severity	Location	Status
Logical Issue	Minor	NCPStaking.sol (v1): 1188	Resolved

Description

The `withdraw()` function includes an argument `to`, which intuitively refers to the recipient of the withdrawn funds. However, a discrepancy is observed in the function's implementation between `withdraw()` and `withdrawAll()`: the funds are sent to `msg.sender` (function caller), rather than the specified `to` address. This inconsistency raises concerns regarding the intended design of the function. Auditors request confirmation from the development team on whether this design is intended.

```
    function withdraw(
        uint256 pid,
        uint256 tokenId,
        address payable to
    ) external nonReentrant whenNotLock(pid) {
        require(pid != 37 && pid != 40, "STAKING: 37, 40 is stoped");
        PoolInfo memory pool = updatePool(pid);
        if (tokenId == 0) {
            tokenId = withdrawalNFT.getFirstWithdrawableToken(pid, msg.sender);
        }
        IWithdrawalNFT.WithdrawalRequestInfo
            memory withdrawalRequestInfo = withdrawalNFT
                .getWithdrawalRequestInfo(tokenId);

        require(
            withdrawalRequestInfo.pid == pid &&
                withdrawalRequestInfo.toPid == pid,
            "STAKING: Invalid token id"
        );
        withdrawalNFT.burn(msg.sender, tokenId);
        payable(address((msg.sender))).sendValue(withdrawalRequestInfo.amount);
        pool.totalRequestedWithdrawal -= withdrawalRequestInfo.amount;
        poolInfo[pid] = pool;
        ...
    }
```

```
function withdrawAll(address payable to) external nonReentrant {
    uint256[] memory withdrawableTokenList = withdrawalNFT
        .getWithdrawableTokenList(msg.sender);
    for (uint256 i = 0; i < withdrawableTokenList.length; i++) {
        IWithdrawalNFT.WithdrawalRequestInfo
            memory withdrawalRequestInfo = withdrawalNFT
                .getWithdrawalRequestInfo(withdrawableTokenList[i]);
        require(
            !poolInfo[withdrawalRequestInfo.pid].lock,
            "STAKING: EMERGENCY!"
        );
        withdrawalNFT.burn(msg.sender, withdrawableTokenList[i]);
        to.sendValue(withdrawalRequestInfo.amount);
        uint256 pid = withdrawalRequestInfo.pid;
        require(pid != 37 && pid != 40, "STAKING: 37, 40 is stoped");
        PoolInfo memory pool = updatePool(pid);
        pool.totalRequestedWithdrawal -= withdrawalRequestInfo.amount;
        poolInfo[pid] = pool;
        ...
}
```

Recommendation

It's recommended that the team make the `to` address of `withdraw()` and `withdrawAll()` the same as `msg.sender`

Alleviation

Issue resolved. Changes have been reflected in the commit hash: https://github.com/wemixarchive/Wonder-staking_private/commit/90399a33c720a6339f152f25f37028d4517b5e5b

RWB-02 | MISSING VALIDATION FOR REWARDER EXISTANCE

Category	Severity	Location	Status
Logical Issue	Minor	Rewarder.sol (v1): 86~96	Resolved

Description

In the current implementation, functions `addRewarder` and `removeRewarder()` do not check if the input parameter is a valid rewarder.

Removing non-exist rewarder will lead to inaccurate `rewarderCount` value.

Recommendation

We recommend implementing proper checks to validate the input parameter `_rewarder` .

Alleviation

Issue resolved. Changes have been reflected in the commit hash: https://github.com/wemixarchive/Wonder-staking_private/commit/90399a33c720a6339f152f25f37028d4517b5e5b

SIB-01 | USAGE OF `transfer()`

Category	Severity	Location	Status
Volatile Code	Minor	StakingImp.sol (v1): 131	Resolved

Description

After EIP-1884 was included in the Istanbul hard fork, it is not recommended to use `.transfer()` or `.send()` for transferring ether as these functions have a hard-coded value for gas costs making them obsolete as they are forwarding a fixed amount of gas, specifically `2300`. This can cause issues in case the linked statements are meant to be able to transfer funds to other contracts instead of EOAs.

Recommendation

We advise that the linked `.transfer()` and `.send()` calls are substituted with the utilization of the `sendValue()` function from the `Address.sol` implementation of OpenZeppelin either by directly importing the library or copying the linked code.

Alleviation

The client revised the code and resolved this issue in commit : 0f582cc48fcbb2a42ba7bca1ebba9ee3d0607cbf

GIB-01 `nonReentrant` MODIFIER CAN BE PLACED AND EXECUTED BEFORE OTHER MODIFIERS IN FUNCTIONS

Category	Severity	Location	Status
Volatile Code	Informational	GovImp.sol (v1): 484~489	Resolved

Description

As a best practice, the `nonReentrant` modifier could be placed and executed before other modifiers in functions to prevent reentrancies through other modifiers and make code more efficient. To follow the best practice, please consider placing the `nonReentrant` modifier before the **onlyGovMem** modifier in the following functions.

Recommendation

Consider placing he `nonReentrant` modifier before other modifiers.

Alleviation

The client revised the code and resolved this issue in commit : ef31b9a97dad3b5cdd3470b9bf16c25295e9ffd4

OPTIMIZATIONS

WEMIX RETAINER 83 GOVERNANCE & WONDER STAKING

ID	Title	Category	Severity	Status
NCS-03	Lack Of Array Duplication Check	Logical Issue	Optimization	Resolved
NCW-08	Unused Mapping	Code Optimization	Optimization	Acknowledged
RWB-01	Unused State Variable	Coding Issue	Optimization	Acknowledged

NCS-03 | LACK OF ARRAY DUPLICATION CHECK

Category	Severity	Location	Status
Logical Issue	Optimization	gateway/NCPStakingGateway.sol (v1): 617	Resolved

Description

The pointed input parameters are not validated. In particular, the following checks should be added:

Check if `newPid` already exists in `pids`.

Recommendation

We recommend adding proper input validation.

Alleviation

Issue resolved. Changes have been reflected in the commit hash: https://github.com/wemixarchive/Wonder-staking_private/commit/90399a33c720a6339f152f25f37028d4517b5e5b

NCW-08 | UNUSED MAPPING

Category	Severity	Location	Status
Code Optimization	Optimization	NCPStaking.sol (v1): 1558	Acknowledged

Description

The mapping `pidMap` is never used in the `NCPStaking.sol`.

```
1558      mapping(uint256 /* pid */ => uint256 /* new pid */) public pidMap;
```

Recommendation

We recommend the team removing unnecessary mapping from the codebase.

RWB-01 | UNUSED STATE VARIABLE

Category	Severity	Location	Status
Coding Issue	Optimization	Rewarder.sol (v1): 18	Acknowledged

Description

Some state variables are not used in the codebase. This can lead to incomplete functionality or potential vulnerabilities if these variables are expected to be utilized.

```
18      uint256 public accRewardPerShare;
```

Recommendation

It is recommended to ensure that all necessary state variables are used, and remove redundant variables.

FORMAL VERIFICATION

WEMIX RETAINER 83 GOVERNANCE & WONDER STAKING

Formal guarantees about the behavior of smart contracts can be obtained by reasoning about properties relating to the entire contract (e.g. contract invariants) or to specific functions of the contract. Once such properties are proven to be valid, they guarantee that the contract behaves as specified by the property. As part of this audit, we applied formal verification to prove that important functions in the smart contracts adhere to their expected behaviors.

Considered Functions And Scope

In the following, we provide a description of the properties that have been used in this audit. They are grouped according to the type of contract they apply to.

Verification of Standard ERC-721 Properties

Properties for standard ERC-721 contracts (note that `safeTransferFrom` function is not included in the verification):

Property Name	Title
erc721common-ownerof-succeed-normal	`ownerOf` Succeeds For Valid Tokens
erc721common-ownerof-no-change-state	`ownerOf` Does Not Change the Contract's State
erc721common-balanceof-revert	`balanceOf` Fails on the Zero Address
erc721common-balanceof-correct-count	`balanceOf` Returns the Correct Value
erc721common-balanceof-succeed-normal	`balanceOf` Succeeds on Valid Inputs
erc721common-balanceof-no-change-state	`balanceOf` Does Not Change the Contract's State
erc721common-transferfrom-revert-invalid	`transferFrom` Fails for Invalid Tokens
erc721common-transferfrom-correct-state-approval	`transferFrom` Has Expected Approval Changes
erc721common-transferfrom-correct-state-owner	`transferFrom` Has Expected Ownership Changes
erc721common-transferfrom-correct-state-balance	`transferFrom` Keeps Balances Constant Except for From and To
erc165-supportsinterface-no-change-state	`supportsInterface` Does Not Change the Contract's State
erc721common-transferfrom-correct-owner-to	`transferFrom` Transfers Ownership
erc721common-transferfrom-correct-balance	`transferFrom` Sum of Balances is Constant

Property Name	Title
erc721common-transferfrom-correct-increase	`transferFrom` Transfers the Complete Token in Transfers
erc721-transferfrom-succeed-normal	`transferFrom` Succeeds on Valid Inputs
erc721common-setapprovalforall-change-state	`setApprovalForAll` Has No Unexpected State Changes
erc721common-setapprovalforall-multiple	`setApprovalForAll` Can Set Multiple Operators
erc721common-transferfrom-correct-approval	`transferFrom` Updates the Approval Correctly
erc721common-setapprovalforall-succeed-normal	`setApprovalForAll` Returns for Valid Inputs
erc721common-setapprovalforall-set-correct	`setApprovalForAll` Approves Operator
erc721common-approve-revert-invalid-token	`approve` Fails For Calls with Invalid Tokens
erc721common-approve-revert-not-allowed	`approve` Prevents Unpermitted Approvals
erc721common-approve-set-correct	`approve` Sets Approval
erc721common-approve-succeed-normal	`approve` Returns for Valid Inputs
erc721common-isapprovedforall-change-state	`isApprovedForAll` Does Not Change the Contract's State
erc721common-isapprovedforall-correct	`isApprovedForAll` Returns Correct Approvals
erc721common-isapprovedforall-succeed	`isApprovedForAll` Always Succeeds
erc721common-approve-change-state	`approve` Has No Unexpected State Changes
erc721common-getapproved-revert-zero	`getApproved` Fails on Invalid Tokens
erc721common-getapproved-correct-value	`getApproved` Returns Correct Approved Address
erc721common-getapproved-succeed-normal	`getApproved` Succeeds For Valid Tokens
erc721common-getapproved-change-state	`getApproved` Does Not Change the Contract's State
erc721common-ownerof-revert	`ownerOf` Fails On Invalid Tokens
erc721common-ownerof-correct-owner	`ownerOf` Returns the Correct Owner
erc721common-supportsinterface-metadata	`supportsInterface` Signals that ERC721Metadata is Implemented
erc721common-supportsinterface-correct-erc721	`supportsInterface` Signals Support for `ERC721`

Property Name	Title
erc165-supportsinterface-correct-false	`supportsInterface` Returns `False` for Id 0xffffffff
erc165-supportsinterface-correct-erc165	`supportsInterface` Signals Support for ERC165
erc165-supportsinterface-succeed-always	`supportsInterface` Always Succeeds
erc721common-transferfrom-revert-exceed-approval	`transferFrom` Fails for Token Transfers without Approval
erc721common-transferfrom-revert-not-owned	`transferFrom` Fails if `From` Is Not Token Owner
erc721common-transferfrom-revert-zero-argument	`transferFrom` Fails for Transfers with Zero Address Arguments

Verification Results

In the remainder of this section, we list all contracts where formal verification of at least one property was not successful. There are several reasons why this could happen:

- False: The property is violated by the project.
- Inconclusive: The proof engine cannot prove or disprove the property due to timeouts or exceptions.
- Inapplicable: The property does not apply to the project.

Detailed Results For Contract WithdrawalNFT (contracts/withdrawalToken/WithdrawalNFT.sol) In Commit 80ec4f9dea8537b30c286c44bbfb1c7ca195e24c

Verification of Standard ERC-721 Properties

Detailed Results for Function `ownerOf`

Property Name	Final Result	Remarks
erc721common-ownerof-succeed-normal	Inconclusive	
erc721common-ownerof-no-change-state	True	
erc721common-ownerof-revert	Inconclusive	
erc721common-ownerof-correct-owner	Inconclusive	

Detailed Results for Function `balanceOf`

Property Name	Final Result	Remarks
erc721common-balanceof-revert	True	
erc721common-balanceof-correct-count	True	
erc721common-balanceof-succeed-normal	True	
erc721common-balanceof-no-change-state	True	

Detailed Results for Function `transferFrom`

Property Name	Final Result	Remarks
erc721common-transferfrom-revert-invalid	Inconclusive	
erc721common-transferfrom-correct-state-approval	True	
erc721common-transferfrom-correct-state-owner	Inconclusive	
erc721common-transferfrom-correct-state-balance	True	
erc721common-transferfrom-correct-owner-to	Inconclusive	
erc721common-transferfrom-correct-balance	True	
erc721common-transferfrom-correct-increase	Inconclusive	
erc721-transferfrom-succeed-normal	Inconclusive	
erc721common-transferfrom-correct-approval	True	
erc721common-transferfrom-revert-exceed-approval	True	
erc721common-transferfrom-revert-not-owned	Inconclusive	
erc721common-transferfrom-revert-zero-argument	True	

Detailed Results for Function `supportsInterface`

Property Name	Final Result	Remarks
erc165-supportsinterface-no-change-state	True	
erc721common-supportsinterface-metadata	True	
erc721common-supportsinterface-correct-erc721	True	
erc165-supportsinterface-correct-false	True	
erc165-supportsinterface-correct-erc165	True	
erc165-supportsinterface-succeed-always	True	

Detailed Results for Function `setApprovalForAll`

Property Name	Final Result	Remarks
erc721common-setapprovalforall-change-state	Inapplicable	The property does not apply to the contract
erc721common-setapprovalforall-multiple	True	
erc721common-setapprovalforall-succeed-normal	True	
erc721common-setapprovalforall-set-correct	True	

Detailed Results for Function `approve`

Property Name	Final Result	Remarks
erc721common-approve-revert-invalid-token	Inconclusive	
erc721common-approve-revert-not-allowed	Inconclusive	
erc721common-approve-set-correct	Inconclusive	
erc721common-approve-succeed-normal	Inconclusive	
erc721common-approve-change-state	True	

Detailed Results for Function `isApprovedForAll`

Property Name	Final Result	Remarks
erc721common-isapprovedforall-change-state	True	
erc721common-isapprovedforall-correct	True	
erc721common-isapprovedforall-succeed	True	

Detailed Results for Function `getApproved`

Property Name	Final Result	Remarks
erc721common-getapproved-revert-zero	Inconclusive	
erc721common-getapproved-correct-value	True	
erc721common-getapproved-succeed-normal	Inconclusive	
erc721common-getapproved-change-state	True	

APPENDIX | WEMIX RETAINER 83 GOVERNANCE & WONDER STAKING

Finding Categories

Categories	Description
Coding Issue	Coding Issue findings are about general code quality including, but not limited to, coding mistakes, compile errors, and performance issues.
Incorrect Calculation	Incorrect Calculation findings are about issues in numeric computation such as rounding errors, overflows, out-of-bounds and any computation that is not intended.
Volatile Code	Volatile Code findings refer to segments of code that behave unexpectedly on certain edge cases and may result in vulnerabilities.
Logical Issue	Logical Issue findings indicate general implementation issues related to the program logic.
Centralization	Centralization findings detail the design choices of designating privileged roles or other centralized controls over the code.
Design Issue	Design Issue findings indicate general issues at the design level beyond program logic that are not covered by other finding categories.

Checksum Calculation Method

The "Checksum" field in the "Audit Scope" section is calculated as the SHA-256 (Secure Hash Algorithm 2 with digest size of 256 bits) digest of the content of each file hosted in the listed source repository under the specified commit.

The result is hexadecimal encoded and is the same as the output of the Linux "sha256sum" command against the target file.

Details on Formal Verification

Some Solidity smart contracts from this project have been formally verified. Each such contract was compiled into a mathematical model that reflects all its possible behaviors with respect to the property. The model takes into account the semantics of the Solidity instructions found in the contract. All verification results that we report are based on that model.

The following assumptions and simplifications apply to our model:

- Certain low-level calls and inline assembly are not supported and may lead to a contract not being formally verified.
- We model the semantics of the Solidity source code and not the semantics of the EVM bytecode in a compiled contract.

Formalism for property specifications

All properties are expressed in a behavioral interface specification language that CertiK has developed for Solidity, which allows us to specify the behavior of each function in terms of the contract state and its parameters and return values, as well as contract properties that are maintained by every observable state transition. Observable state transitions occur when the contract's external interface is invoked and the invocation does not revert, and when the contract's Ether balance is changed by the EVM due to another contract's "self-destruct" invocation. The specification language has the usual Boolean connectives, as well as the operator `\old` (used to denote the state of a variable before a state transition), and several types of specification clause:

Apart from the Boolean connectives and the modal operators "always" (written `[]`) and "eventually" (written `<>`), we use the following predicates to reason about the validity of atomic propositions. They are evaluated on the contract's state whenever a discrete time step occurs:

- `requires [cond]` - the condition `cond`, which refers to a function's parameters, return values, and contract state variables, must hold when a function is invoked in order for it to exhibit a specified behavior.
- `ensures [cond]` - the condition `cond`, which refers to a function's parameters, return values, and both `\old` and current contract state variables, is guaranteed to hold when a function returns if the corresponding requires condition held when it was invoked.
- `invariant [cond]` - the condition `cond`, which refers only to contract state variables, is guaranteed to hold at every observable contract state.
- `constraint [cond]` - the condition `cond`, which refers to both `\old` and current contract state variables, is guaranteed to hold at every observable contract state except for the initial state after construction (because there is no previous state); constraints are used to restrict how contract state can change over time.

Description of the Analyzed ERC-721 Properties

Properties related to function `ownerOf`

erc721common-ownerof-correct-owner

Invocations of `ownerOf(token)` must return the owner for a valid token `token` that is held in the contract's owner mapping.

Specification:

```
requires ownerOf(tokenId) != address(0);
ensures \result == \old(ownerOf(tokenId));
```

erc721common-ownerof-no-change-state

Function `ownerOf` must not change any of the contract's state variables.

Specification:

```
assignable \nothing;
```

erc721common-ownerof-revert

Invocations of `ownerOf(token)` must fail for an invalid token.

Specification:

```
reverts_when ownerOf(tokenId) == address(0);
```

erc721common-ownerof-succeed-normal

Function `ownerOf(token)` must always succeed for valid tokens if it does not run out of gas.

Specification:

```
requires ownerOf(tokenId) != address(0);
ensures true;
reverts_only_when false;
```

Properties related to function `balanceOf`

erc721common-balanceof-correct-count

Invocations of `balanceOf(owner)` must return the value that is held in the balance mapping for address `owner`.

Specification:

```
ensures \result == \old(balanceOf(owner));
```

erc721common-balanceof-no-change-state

Function `balanceOf` must not change any of the contract's state variables.

Specification:

```
assignable \nothing;
```

erc721common-balanceof-revert

Invocations of `balanceOf(owner)` must fail if the address `owner` is the zero address.

Specification:

```
reverts_when owner == address(0);
```

erc721common-balanceof-succeed-normal

All invocations of `balanceOf(owner)` must succeed if the address `owner` is not zero and it does not run out of gas.

Specification:

```
requires owner != address(0);
ensures true;
reverts_only_when false;
```

Properties related to function `transferFrom`

erc721-transferfrom-succeed-normal

All invocations of `transferFrom(from, to, tokenId)` must succeed if

- address `from` is the owner of token `tokenId` , *it is not a self transfer,
- the sender is approved to transfer token `tokenId` ,
- transferring the token to the address `to` does not lead to an overflow of the recipient's balance, and
- the supplied gas suffices to complete the call.

Specification:

```
requires from != address(0);
requires to != address(0);
requires from != to;
requires from == ownerOf(tokenId);
requires balanceOf(from) > 0;
requires balanceOf(to) < type(uint256).max;
requires (msg.sender == from)||(getApproved(tokenId) == msg.sender) ||
isApprovedForAll(from,msg.sender);
ensures true;
reverts_only_when false;
```

erc721common-transferfrom-correct-approval

All non-reverting invocations of `transferFrom(from, to, tokenId)` that return must remove any approval for token `tokenId` .

Specification:

```
ensures getApproved(tokenId) == address(0);
```

erc721common-transferfrom-correct-balance

All non-reverting invocations of `transferFrom(from, to, tokenId)` must keep the sum of token balances constant.

Specification:

```
requires from != address(0);
requires to != address(0);
requires balanceOf(from) > 0;
requires balanceOf(to) < type(uint256).max;
ensures (\old(balanceOf(from)) - balanceOf(from)) == (balanceOf(to) -
\old(balanceOf(to)));
```

erc721common-transferfrom-correct-increase

All invocations of `transferFrom(from, to, tokenId)` that succeed must subtract a token from the balance of address `from` and add the token to the balance of address `to`.

Specification:

```
requires from != to;
requires balanceOf(from) > 0;
requires balanceOf(to) < type(uint256).max;
ensures (balanceOf(from) == \old(balanceOf(from)) - 1) && (balanceOf(to) ==
\old(balanceOf(to)) + 1);
  also
requires from == to;
requires ownerOf(tokenId) == from;
ensures balanceOf(from) == \old(balanceOf(from));
```

erc721common-transferfrom-correct-owner-to

All non-reverting invocations of `transferFrom(from, to, tokenId)` must transfer the ownership of token `tokenId` to the address `to`.

Specification:

```
requires from != address(0);
requires to != address(0);
requires (msg.sender == from) || (getApproved(tokenId) == msg.sender) ||
isApprovedForAll(from,msg.sender);
ensures ownerOf(tokenId) == to;
```

erc721common-transferfrom-correct-state-approval

All non-reverting invocations of `transferFrom(from, to, tokenId)` must remove only approvals for token `tokenId`

Specification:

```
ensures (\forall uint id | id!=tokenId :: \old(getApproved(id))==getApproved(id));
```

erc721common-transferfrom-correct-state-balance

All non-reverting invocations of `transferFrom(from, to, tokenId)` must only modify the balance of the addresses `from` and `to`.

Specification:

```
ensures (\forall address adr | (adr!=from) && (adr!=to) :: \old(balanceOf(adr))==
balanceOf(adr));
```

erc721common-transferfrom-correct-state-owner

All non-reverting invocations of `transferFrom(from, to, tokenId)` must only modify the ownership of token `tokenId`.

Specification:

```
ensures (\forall uint id | id!=tokenId :: \old(ownerOf(id))==ownerOf(id));
```

erc721common-transferfrom-revert-exceed-approval

Any call of the form `transferFrom(from, to, tokenId)` must fail if the sender is neither the token owner nor an operator of the token owner nor approved for token `tokenId`.

Specification:

```
reverts_when (msg.sender != from) && (getApproved(tokenId) != msg.sender) &&
!isApprovedForAll(from,msg.sender);
```

erc721common-transferfrom-revert-invalid

All calls of the form `transferFrom(from, to, tokenId)` must fail for any invalid token.

Specification:

```
reverts_when ownerOf(tokenId) == address(0);
```

erc721common-transferfrom-revert-not-owned

Any call of the form `transferFrom(from, to, tokenId)` must fail if address 'from' is not the owner of token `tokenId`.

Specification:

```
reverts_when ownerOf(tokenId) != from;
```

erc721common-transferfrom-revert-zero-argument

All calls of the form `transferFrom(from, to, tokenId)` must fail for transfers from or to the zero address.

Specification:

```
reverts_when to == address(0);
also
reverts_when from == address(0);
```

Properties related to function `supportsInterface`

erc165-supportsinterface-correct-erc165

Invocations of `supportsInterface(id)` must signal that the interface `ERC165` is implemented.

Specification:

```
requires interfaceId == 0x01ffc9a7;
ensures \result;
```

erc165-supportsinterface-correct-false

Invocations of `supportsInterface(id)` with `id` 0xffffffff must return `false`.

Specification:

```
requires interfaceId == 0xffffffff;
ensures !\result;
```

erc165-supportsinterface-no-change-state

Function `supportsInterface` must not change any of the contract's state variables.

Specification:

```
assignable \nothing;
```

erc165-supportsinterface-succeed-always

Function `supportsInterface` must always succeed if it does not run out of gas.

Specification:

```
reverts_only_when false;
```

erc721common-supportsinterface-correct-erc721

Invocations of `supportsInterface(id)` must signal that the interface `ERC721` is implemented.

Specification:

```
requires interfaceId == 0x80ac58cd;
ensures esult;
```

erc721common-supportsinterface-metadata

A call of `supportsInterface(interfaceId)` with the interface id of ERC721Metadata must return true.

Specification:

```
requires interfaceId == 0x5b5e139f;
ensures \result;
```

Properties related to function `setApprovalForAll`

erc721common-setapprovalforall-change-state

All calls of the form `setApprovalForAll(operator, approved)` must only update the approval mapping according to the message sender, the address `operator` and the Boolean value `approved` but incur no other state changes.

Specification:

```
assignable isApprovedForAll(msg.sender,operator);
```

erc721common-setapprovalforall-multiple

Calls of the form `setApprovalForAll(operator, approved)` must be able to set multiple operators for the tokens of the message sender.

Specification:

```
requires approved;
ensures isApprovedForAll(msg.sender,operator);
ensures (\forall address op1 |(op1 != address(0)) &&
\old(isApprovedForAll(msg.sender,op1)) :: isApprovedForAll(msg.sender,op1));
```

erc721common-setapprovalforall-set-correct

All non-reverting calls of the form `setApprovalForAll(operator, approved)` must set the approval of a non-zero address `operator` according to the Boolean value `approved`.

Specification:

```
requires operator != address(0);
ensures isApprovedForAll(msg.sender,operator) == approved;
```

erc721common-setapprovalforall-succeed-normal

Calls of the form `setApprovalForAll(operator, approved)` must return if

- the message sender is not the `operator`,
- `operator` is not the zero address and
- the execution does not run out of gas.

Specification:

```
requires msg.sender != operator;
requires operator != address(0);
ensures true;
reverts_only_when false;
```

Properties related to function `approve`

erc721common-approve-change-state

All calls of the form `approve(to, tokenId)` must only update the allowance mapping according to a valid token `tokenId` and the address `to`, and incur no other state changes.

Specification:

```
assignable getApproved(tokenId);
```

erc721common-approve-revert-invalid-token

All calls of the form `approve(to, tokenId)` must fail for an invalid token.

Specification:

```
reverts_when ownerOf(tokenId) == address(0);
```

erc721common-approve-revert-not-allowed

All calls of the form `approve(to, tokenId)` must fail if the message sender is not permitted to access token `tokenId`.

Specification:

```
reverts_when (ownerOf(tokenId) != msg.sender) &&
!isApprovedForAll(ownerOf(tokenId),msg.sender);
```

erc721common-approve-set-correct

Any returning call of the form `approve(to, tokenId)` must approve the address `to` for token `tokenId`.

Specification:

```
requires ownerOf(tokenId) != address(0);
requires (ownerOf(tokenId) == msg.sender)||
isApprovedForAll(ownerOf(tokenId),msg.sender);
ensures getApproved(tokenId) == to;
```

erc721common-approve-succeed-normal

All calls of the form `approve(to, tokenId)` must return if

- the sender is the owner or an authorized operator of the owner
- the token `tokenId` is valid and
- the execution does not run out of gas.

Specification:

```
requires ownerOf(tokenId) != address(0);
requires ownerOf(tokenId) != to;
requires (ownerOf(tokenId) == msg.sender) ||
isApprovedForAll(ownerOf(tokenId),msg.sender);
ensures true;
reverts_only_when false;
```

Properties related to function `isApprovedForAll`

erc721common-isapprovedforall-change-state

Function `isApprovedForAll` does not change any of the contract's state variables.

Specification:

```
assignable \nothing;
```

erc721common-isapprovedforall-correct

Invocations of `isApprovedForAll(owner, operator)` must return whether a non-zero address `operator` is approved for tokens of a non-zero address `owner`, or return false.

Specification:

```
requires owner != address(0);
requires operator != address(0);
ensures \result == isApprovedForAll(owner,operator);
```

erc721common-isapprovedforall-succeed

Function `isApprovedForAll` does always succeed, assuming that its execution does not run out of gas.

Specification:

```
reverts_only_when false;
```

Properties related to function `getApproved`

erc721common-getapproved-change-state

Function `getApproved` must not change any of the contract's state variables.

Specification:

```
assignable \nothing;
```

erc721common-getapproved-correct-value

Invocations of `getApproved(token)` must return the approved address of a valid `token`.

Specification:

```
ensures (\result == \old(getApproved(tokenId))) || (\result == address(0));
```

erc721common-getapproved-revert-zero

Invocations of `getApproved(token)` with an invalid token must fail.

Specification:

```
reverts_when ownerOf(tokenId) == address(0);
```

erc721common-getapproved-succeed-normal

Function `getApproved` must always succeed for valid tokens, assuming that its execution does not run out of gas.

Specification:

```
requires ownerOf(tokenId) != address(0);
ensures true;
reverts_only_when false;
```

DISCLAIMER | CERTIK

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability) set forth in the Services Agreement, or the scope of services, and terms and conditions provided to you ("Customer" or the "Company") in connection with the Agreement. This report provided in connection with the Services set forth in the Agreement shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes, nor may copies be delivered to any other person other than the Company, without CertiK's prior written consent in each instance.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts CertiK to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. CertiK's position is that each company and individual are responsible for their own due diligence and continuous security. CertiK's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by CertiK is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

ALL SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF ARE PROVIDED "AS IS" AND "AS AVAILABLE" AND WITH ALL FAULTS AND DEFECTS WITHOUT WARRANTY OF ANY KIND. TO THE MAXIMUM EXTENT PERMITTED UNDER APPLICABLE LAW, CERTIK HEREBY DISCLAIMS ALL WARRANTIES, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE WITH RESPECT TO THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS. WITHOUT LIMITING THE FOREGOING, CERTIK SPECIFICALLY DISCLAIMS ALL IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE AND NON-INFRINGEMENT, AND ALL WARRANTIES ARISING FROM COURSE OF DEALING, USAGE, OR TRADE PRACTICE. WITHOUT LIMITING THE FOREGOING, CERTIK MAKES NO WARRANTY OF ANY KIND THAT THE SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF, WILL MEET CUSTOMER'S OR ANY OTHER PERSON'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULT, BE COMPATIBLE OR WORK WITH ANY SOFTWARE, SYSTEM, OR OTHER SERVICES, OR BE SECURE, ACCURATE, COMPLETE, FREE OF HARMFUL CODE, OR ERROR-FREE. WITHOUT LIMITATION TO THE FOREGOING, CERTIK PROVIDES NO WARRANTY OR

UNDERTAKING, AND MAKES NO REPRESENTATION OF ANY KIND THAT THE SERVICE WILL MEET CUSTOMER'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULTS, BE COMPATIBLE OR WORK WITH ANY OTHER SOFTWARE, APPLICATIONS, SYSTEMS OR SERVICES, OPERATE WITHOUT INTERRUPTION, MEET ANY PERFORMANCE OR RELIABILITY STANDARDS OR BE ERROR FREE OR THAT ANY ERRORS OR DEFECTS CAN OR WILL BE CORRECTED.

WITHOUT LIMITING THE FOREGOING, NEITHER CERTIK NOR ANY OF CERTIK'S AGENTS MAKES ANY REPRESENTATION OR WARRANTY OF ANY KIND, EXPRESS OR IMPLIED AS TO THE ACCURACY, RELIABILITY, OR CURRENCY OF ANY INFORMATION OR CONTENT PROVIDED THROUGH THE SERVICE. CERTIK WILL ASSUME NO LIABILITY OR RESPONSIBILITY FOR (I) ANY ERRORS, MISTAKES, OR INACCURACIES OF CONTENT AND MATERIALS OR FOR ANY LOSS OR DAMAGE OF ANY KIND INCURRED AS A RESULT OF THE USE OF ANY CONTENT, OR (II) ANY PERSONAL INJURY OR PROPERTY DAMAGE, OF ANY NATURE WHATSOEVER, RESULTING FROM CUSTOMER'S ACCESS TO OR USE OF THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS.

ALL THIRD-PARTY MATERIALS ARE PROVIDED "AS IS" AND ANY REPRESENTATION OR WARRANTY OF OR CONCERNING ANY THIRD-PARTY MATERIALS IS STRICTLY BETWEEN CUSTOMER AND THE THIRD-PARTY OWNER OR DISTRIBUTOR OF THE THIRD-PARTY MATERIALS.

THE SERVICES, ASSESSMENT REPORT, AND ANY OTHER MATERIALS HEREUNDER ARE SOLELY PROVIDED TO CUSTOMER AND MAY NOT BE RELIED ON BY ANY OTHER PERSON OR FOR ANY PURPOSE NOT SPECIFICALLY IDENTIFIED IN THIS AGREEMENT, NOR MAY COPIES BE DELIVERED TO, ANY OTHER PERSON WITHOUT CERTIK'S PRIOR WRITTEN CONSENT IN EACH INSTANCE.

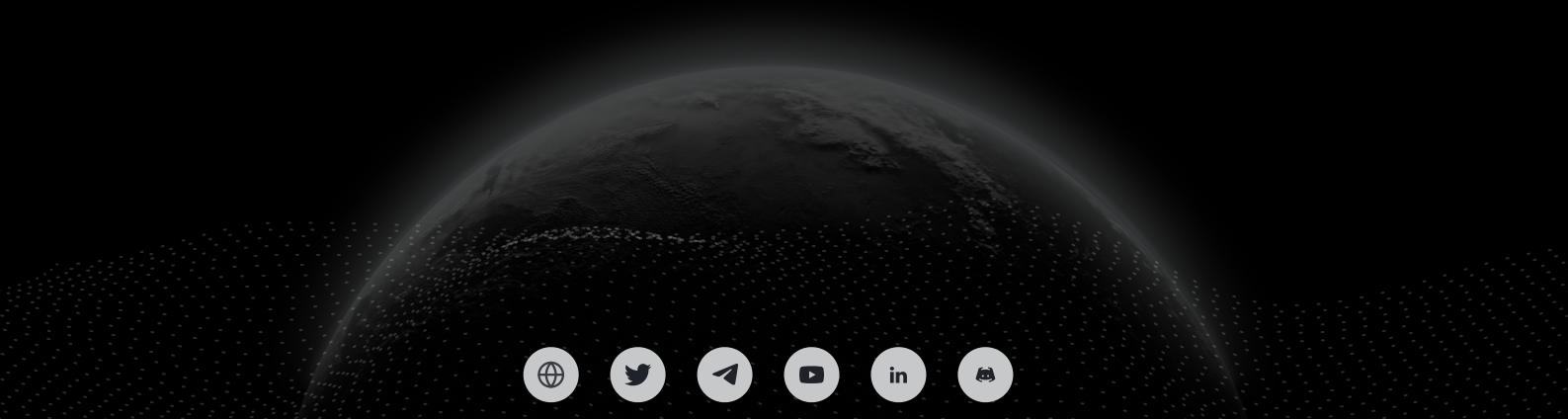
NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS.

THE REPRESENTATIONS AND WARRANTIES OF CERTIK CONTAINED IN THIS AGREEMENT ARE SOLELY FOR THE BENEFIT OF CUSTOMER. ACCORDINGLY, NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH REPRESENTATIONS AND WARRANTIES AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH REPRESENTATIONS OR WARRANTIES OR ANY MATTER SUBJECT TO OR RESULTING IN INDEMNIFICATION UNDER THIS AGREEMENT OR OTHERWISE.

FOR AVOIDANCE OF DOUBT, THE SERVICES, INCLUDING ANY ASSOCIATED ASSESSMENT REPORTS OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.

CertiK | Securing the Web3 World

Founded in 2017 by leading academics in the field of Computer Science from both Yale and Columbia University, CertiK is a leading blockchain security company that serves to verify the security and correctness of smart contracts and blockchain-based protocols. Through the utilization of our world-class technical expertise, alongside our proprietary, innovative tech, we're able to support the success of our clients with best-in-class security, all whilst realizing our overarching vision; provable trust for all throughout all facets of blockchain.



Copyright © CertiK